Enhancing AIoT Device Association With Task Offloading in Aerial MEC Networks

Jingxuan Chen, Peng Yang, Siqiao Ren, Zhongliang Zhao, Xianbin Cao, *Senior Member, IEEE*, and Dapeng Wu, *Fellow, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs) have emerged as a promising solution for enhancing mobile-edge computing (MEC) networks. However, the integration of UAVs into MEC networks poses unique challenges, such as the presence of dynamic devices and complex resource allocation. This research investigates the problem of task offloading in a distributed MEC network with multiple ground and aerial base stations (UAV base stations). With a focus on the cost-sensitive nature of Internet of Things Devices (IoTDs), our objective is to maximize the Quality of Experience (QoE) in terms of average task response time and cache queue length in IoTDs by jointly optimizing device association, offloading decision, and UAV trajectory planning. To address the combinatorial and nonconvex nature of the problem, we propose an artificial intelligence (AI)-based optimization scheme. First, the association between IoTDs and stations is determined using a recursive selection and replacement transmission-rate-based (RSRT) algorithm. Subsequently, the offloading problem is formulated as a 0-1 Backpack Problem with variable value, for which we present a backtracking task offloading (BTO) algorithm. Additionally, we employ a multiagent deep deterministic policy gradient (MADDPG) approach to determine the trajectory planning of UAVs. Numerical results demonstrate the effectiveness of the proposed scheme in terms of reduction in average response time, and cache queue length in IoTDs within the MEC system when compared to benchmark schemes.**

***Index Terms*—Device association, MEC, multiagent deep deterministic policy gradient (MADDPG), task offloading, unmanned aerial vehicle (UAV).**

Manuscript received 15 May 2023; revised 2 July 2023; accepted 19 July 2023. Date of publication 1 August 2023; date of current version 25 December 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61827901; in part by the Application Research of UAV Inspection System in Facilities and Bridges of Shuohuang Railway under Grant GJNY-19-90; and in part by the National Key Research and Development Program of China under Grant 2021YFB3901500. *(Corresponding authors: Zhongliang Zhao; Xianbin Cao.)*

Jingxuan Chen and Siqiao Ren are with the School of Electronic and Information Engineering and the Key Laboratory of Advanced Technology of Near Space Information System, Ministry of Industry and Information Technology of China, Beihang University, Beijing 100191, China (e-mail: chenjingxuan@buaa.edu.cn; rsq0227@buaa.edu.cn).

Peng Yang, Zhongliang Zhao, and Xianbin Cao are with the School of Electronic and Information Engineering and the Key Laboratory of Advanced Technology of Near Space Information System, Ministry of Industry and Information Technology of China, Beihang University, Beijing 100191, China, and also with the Department of Mathematics and Theories, Peng Cheng Laboratory, Shenzhen 518055, Guangdong, China (e-mail: yangp09@buaa.edu.cn; zhaozl@buaa.edu.cn; xbcao@buaa.edu.cn).

Dapeng Wu is with the Department of Computer Science, City University of Hong Kong, Hong Kong (e-mail: dpwu@ieee.org).

Digital Object Identifier 10.1109/JIOT.2023.3300011

I. Introduction

IN CONTRAST to cloud computing, edge computing is situated in closer proximity to terminal devices, thereby facilitating superior computing and communication services. Nonetheless, the burgeoning expansion of Internet of Things Devices (IoTDs) has rendered terrestrial infrastructure increasingly inadequate in addressing the demands of numerous wireless applications that necessitate substantial bandwidth [1] and heterogeneous Quality of Service (QoS) [2]. A viable solution to surmount this challenge involves deploying edge servers aerially to bolster ground base stations (GBSs). Moreover, aerial base stations can augment the likelihood of Line-of-Sight (LoS) links, particularly in specialized communication contexts, such as natural disasters and mountainous regions characterized by intricate topography. Consequently, unmanned aerial vehicles (UAVs) have experienced a surge in industrial and commercial adoption, attributable to their inherent flexibility, high mobility, and cost effectiveness.

In 2017, the European Telecommunications Standards Institute (ETSI) rebranded mobile-edge computing as multiaccess edge computing (MEC), which offers efficacious cloud computing capabilities for IoTDs within the access network. Concurrently, UAVs, functioning as MEC servers, have emerged as a crucial means of delivering computing resources to IoTDs with adaptability. Unlike fixed edge servers, the dynamic nature of UAVs necessitates the application of algorithms by researchers to determine the optimal edge server placement [3]. As a result, UAV-assisted MEC networks effectively mitigate the paucity of spectrum and computing resources while reducing access and response time [4].

Conventional cloud-based architectures collect data and perform calculations centrally via cloud servers. However, distributed architectures present a multitude of benefits over centralized computing. In a distributed system, tasks are processed proximate to data sources or end users, engendering reduced latency and expedited response times. This characteristic is especially critical for time-sensitive applications or users dispersed across vast geographical areas. In this study, we examine the task offloading challenge within a distributed MEC network facilitated by multiple UAVs in order to proficiently manage elevated communication latency caused by congestion while concurrently diminishing the communication load.

A. Related Work

A significant portion of the scholarly investigation has been centered on a singular UAV-based MEC network [5], [6], [7], [8]. In a pioneering study, Mukherjee et al. [5] probed the ideal offloading strategy in a standalone UAV-assisted MEC system. Their primary objective was to minimize the weighted sum of delay and energy consumption. Li et al. [6] further broadened this paradigm, emphasizing the maximization of UAV energy efficiency. This was accomplished through the simultaneous optimization of the UAV trajectory, user transmission power, and computation load allocation. The methodology they employed involved the successive convex approximation (SCA) technique coupled with the Dinkelbach algorithm. In contrast, Han et al.'s research [7] shifted its focus to two operational modes: 1) centralized-MEC computation and 2) distributed-MEC computation. They introduced a joint rate splitting problem to optimally distribute rates for transmission links between two antenna arrays on the UAV. In addition, Fragkos et al. [8] proposed a robust data offloading strategy for users. In their theoretical construct, a UAV-MEC hovers over ground-based nodes and leverages alternate reinforcement learning methodologies.

Considering the constraints inherent in a single UAV within a MEC network, particularly the limited capacity for computing and communication, the exploration of multi-UAV task offloading has emerged. However, optimizing the offloading decision while simultaneously considering the trajectory planning of multiple UAVs presents considerable challenges. A number of studies have thus opted to focus solely on offloading policies while maintaining predetermined UAV mobility patterns. These include scenarios where UAVs hover or follow a prescheduled flight path, moving at a fixed speed in a designated direction [9], or UAVs flying at variable speeds within a predefined area [10]. In other instances, UAVs have been depicted as flying in formation along a circular path [11], being uniformly deployed [12], or each UAV hovers over a specific, fixed area [13], [14]. These various settings serve to simplify the offloading decision-making process, yet also highlight the need for further research that fully integrates dynamic UAV movement and task offloading optimization.

In the literature, a variety of approaches have been adopted to tackle the complex issue of UAV path planning and task offloading. For instance, Miao et al. [15] presented a drone-swarm-assisted MEC algorithm, optimizing path planning and task offloading. Prioritizing monitoring areas, drone energy, and target distances, the proposed algorithm maximizes offloading services while enhancing energy efficiency and reducing latency.

In a different study, Qian et al. [16] proposed a top-layered algorithm based on deep reinforcement learning (DRL). Their goal was to minimize the total energy consumption by jointly optimizing several factors: the offloaded workload of the UAVs, the transmit power, the computing resource allocation, and the UAV trajectory. Meanwhile, in [17], a novel DRL technique was introduced, designed to make optimal computation offloading decisions and flight orientation choices concurrently for multi-UAV cooperative target search. Furthermore, Bai et al. [18] put forth TANTO, a pioneering task offloading scheme for Internet of Things (IoT) devices in regions lacking available communication infrastructure. Also, in the study presented by Bai et al. [19], a novel framework termed the intelligent active data processing (IADP) scheme is introduced. The IADP scheme functions by intelligently and proactively disseminating data information. It incorporates active data processing with deadline priority and integrates artificial intelligence (AI)-based UAV trajectory optimization to determine the most efficient transmission paths. This wide array of studies underscores the innovation and diversity of approaches being employed to address the challenges inherent in multi-UAVs-based MEC networks. Chen et al. [20] proposed a low-complexity iterative algorithm for a MEC-aided-UAV system, optimizing UAV trajectory, transmit power, computation ratio, and base station selection, aiming to secure task offloading while considering practical constraints.

However, a fundamental technical challenge inherent in this problem is the dynamic coordination of trajectories and offloading amongst these UAVs, given their computing tasks. The issue is further complicated by the necessity to make dynamic decisions to ensure the system's communication capabilities within a complex environment. Moreover, a centralized architecture, while potentially useful, may introduce a significant degree of communication latency, subsequently impacting the overall efficiency of communication. As a result, to circumvent this issue, some researchers [21], [22], [23], [24] have proposed a distributed architecture. This architectural design aims to resolve the latency issue and bolster communication efficiency, thus addressing the inherent complexities of coordinating multi-UAV operations in MEC networks.

In the academic literature, various studies have proposed solutions to optimize the performance of distributed UAV-aided MEC networks. For instance, Chen et al. [21] developed a multiagent reinforcement learning framework designed to devise a cooperative movement strategy for multiple cache-enabled UAVs, taking user mobility into account. In a different study, Yan et al. [22] introduced RUDDER, an innovative algorithm for data offloading in IoT scenarios where network coverage is absent. RUDDER optimizes UAV scheduling, reducing cache queue length and prolonging UAV operational time. In another research endeavor, Lakew et al. [23] proposed the optimization of computing, communication, and energy resources in aerial access networks, which consist of high-altitude platforms and UAVs. The aim of this optimization is to maximize service satisfaction and minimize energy consumption for IoTDs. To facilitate the challenge, a deep actor–critic algorithm is proposed to demonstrate superior performance and reliable convergence. Yang et al. [24] presented an online algorithm to optimize energy and task processing rates in a UAV-enabled MEC system serving distributed energy-harvesting devices. We summarize the difference between the existing researches and our work in Table I.

B. Motivation and Contribution

The objective of this article is to achieve a tradeoff between response time and cache queue length of IoTDs in accordance with the computing requirements of IoTDs. Our

TABLE I
COMPARISON BETWEEN OUR WORK AND THE EXISTING LITERATURE

Reference	Single UAV	Multi UAV	MEC	Path planning	Offloading decision	Reinforcement learning (e.g.Q-learning)	Multi agent
[5]	✓		✓		✓	✓	
[6]	✓			✓			
[7]	✓	✓	✓				✓
[8]	✓		✓		✓	✓	
[9]		✓	✓		✓	✓	✓
[10]		✓			✓	✓	
[11]		✓	✓			✓	✓
[12]		✓	✓		✓	✓	
[13]		✓	✓		✓		
[14]		✓	✓		✓		
[15]		✓	✓	✓	✓		
[16]		✓		✓	✓	✓	
[17]		✓		✓	✓	✓	
[18]		✓		✓	✓		
[19]		✓	✓	✓			
[20]		✓	✓	✓	✓		
[21]		✓	✓	✓		✓	✓
[22]		✓		✓		✓	✓
[23]		✓	✓		✓		✓
[24]		✓	✓		✓		✓
Our Work		✓	✓	✓	✓	✓	✓

work focuses on enhancing the performance of the overall Quality of Experience (QoE) of the system through the optimization of device association, task offloading, and control of UAV's motion. To the best of our knowledge, the current body of research on multi-UAV MEC networks remains insufficient, particularly in regards to combining task offloading and UAV movement based on DRL. The utilization of DRL-based approaches in multi-UAV MEC networks poses a significant challenge in terms of identifying appropriate targets and balancing the tradeoff between exploration and exploitation. Furthermore, given the highly dynamic nature of UAVs, designing algorithms with low time complexity to meet real-time requirements represents an additional challenge. The main contributions of our work are summarized as follows.

1) This article investigates the optimization problem of maximizing the long-term reward with regard to response time of tasks and cache queue of IoTDs by jointly optimizing device association, task offloading, and UAV movement. Moreover, we incorporate the constraints of energy consumption of UAV movement and MEC server's computing capacity.
2) We present a groundbreaking methodology, designated as recursive selection and replacement transmission-rate-based (RSRT), devised to address the issue of IoT device association. The proposed technique employs a greedy algorithmic principle to prioritize IoT devices exhibiting the greatest probability of attaining optimal channel quality. This method encompasses a recursive process of selecting and replacing IoT devices contingent on their transmission rates, with the objective of augmenting the efficiency and effectiveness of IoT device association procedures. Furthermore, the operations of the residual IoT devices are provisionally retained in a cache queue until the corresponding devices secure adequate communication resources.
3) We formulate a backtracking task offloading (BTO) algorithm characterized by low time complexity. Specifically, we initially redefine the task offloading problem as a 0-1 knapsack problem with variable value. Subsequently, we deduce the priority order of the solution space and devise a pruning strategy in accordance with the constraint conditions. Ultimately, the optimal offloading strategy is acquired through a backtracking process.
4) We apply the multiagent deep deterministic policy gradient (MADDPG) algorithm to UAV movement control. As an autonomous learning agent, each UAV generates a minimal amount of data information, which serves as state training data for reinforcement learning. Moreover, we ascertain the optimal hyperparameters that yield the most favorable model training outcomes through extensive numerical simulations. Subsequently, we juxtapose the RSRT and BTO algorithms with other benchmark algorithms. The simulation outcomes reveal that the proposed algorithm can efficaciously decrease both the response time and cache queue length.

The remainder of this manuscript is structured in the following manner. In Section II, we present the system model. Subsequently, Section III elaborates on the formulation of the optimization problem. In Section IV, we expound upon the specific solution to the aforementioned optimization problem. The efficacy of the proposed algorithm is demonstrated via simulation outcomes in Section V. Finally, we provide concluding remarks in Section VI.

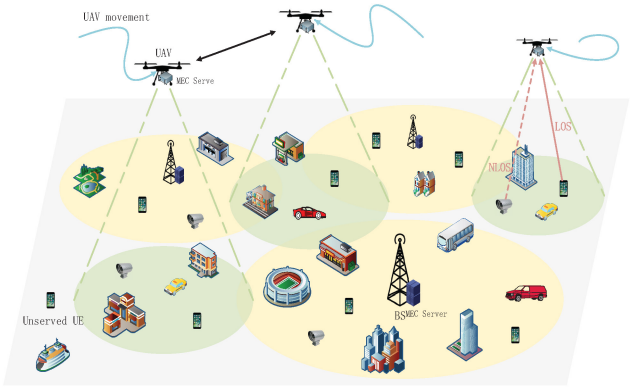


Fig. 1. Enhanced AIoT aerial MEC network scenario.

II. System Model

We consider a MEC network topology, which is consisted of M^b BSs and M^u UAVs underlaid with MEC servers. For brief description, the set of M^b MEC servers on BSs is denoted by $\mathcal{D}_b = \{d_0, \ldots, d_{M_b-1}\}$, and the set of $\mathcal{D}_u$ MEC servers on UAVs is denoted by $\mathcal{D}_u = \{d_{M^b+1}, \ldots, d_{M_b+M_u-1}\}$. Then, the set of all the MEC servers can be denoted by $\mathcal{D} = \mathcal{D}_b \cup \mathcal{D}_u$. N IoTDs, represented by $\mathcal{U} = \{u_0, \ldots, u_{N-1}\}$, are distributed within the geographical area with $L \times L$ square meters. In this scenario, the UAV takes off from its initial position $P_j^{\text{init}} = [x_j^{\text{init}}, y_j^{\text{init}}, 0]$ to serve the IoTDs and eventually returns to P_j^{init} within the total duration T. At time step τ, the positions of d_j and u_i are, respectively, denoted by $\omega_{d_j}(\tau) = [x_{d_j}(\tau), y_{d_j}(\tau), z_{d_j}(\tau)]$ and $\omega_{u_i}(\tau) = [x_{d_j}(\tau), y_{d_j}(\tau), 0]$. Also, let $\mathcal{I}$ and $\mathcal{J}$ represent the set of indexes of $\mathcal{U}$ and $\mathcal{D}$. The considered aerial MEC network scenario is shown in Fig. 1.

The computing task $T_i(t) = \{l_i^t, \phi_i^t\}$ is randomly generated on u_i and stored in the cache queue $Q_i(\tau)$ of length $L_i(\tau)$, where l_i^t and ϕ_i^t represent the amount of task data and the computing intensity (CPU-Cycles/bit), respectively. Different from the current literature that the computation task arrives at u_i with a similar computation intensity [25], [26], [27], we set several levels of computation intensity corresponding to different types of computing task. Moreover, we define t^{opt} as the optimal latency experience. It has the probability that executing the task on local does not satisfy the latency limitation, by the time, the IoTD offloads the fraction of the task queue to the MEC server.

It is worth noting that the computing task is indivisible, namely, the task length cannot be simply divided into two parts with an offloading ratio like [25], [27], and [14]. Thus, we can only offload a subset of each IoTD's task queue at each time step. Let $\mathcal{B}(\tau) = \{\mathcal{B}_0(\tau), \ldots, \mathcal{B}_i(\tau), \ldots, \mathcal{B}_{N-1}(\tau)\}$ represent the offloading strategy at time step τ, and $\mathcal{B}_i(\tau) = \{\beta_i^0(\tau), \ldots, \beta_i^t(\tau), \ldots, \beta_i^\tau(\tau)\}, t \in [0, \tau]$, where $\beta_i^t(\tau) \in \{0, 1\}$ indicates whether u_i offloads the computation task $T_i(t)$ at time step τ. We further denote by $L_i(\tau)$ the cache queue length of u_i with the maximum length $L_i^{\max}$. When $L_i(\tau - 1) + l_i(\tau) > L_i^{\max}$, the computation task $T_i(\tau)$ is lost and unrecoverable.

A. UAV Movement Model

At time step τ, the UAV with MEC server d_j mounted adaptively generates a movement strategy $\mathcal{M}_j(\tau) = \{m_j(\tau), \alpha_j(\tau), \theta_j(\tau)\}(j \geq M_b)$, which is characterized by: 1) $\mathcal{M}_j(\tau)$ is a vector in 3-D polar coordinates; 2) due to the limitations of battery life and flying speed of UAVs, we set a constant $v_j^{\max}$ to present the maximum flying speed (we consider the calculation of maximum movement distance $m_j^{\max}(\tau)$ in subsequent); and 3) UAV d_j flies in a direction with the azimuth angle $\alpha_j(\tau) \in [0, 2\pi)$ and the pitch angle $\theta_j(\tau) \in [-\pi, \pi]$ at time step τ. Therefore, the 3-D position coordinates of the UAV d_j are updated as follows:

$$\begin{aligned} x_{d_j}(\tau+1) &= x_{d_j}(\tau) + m_j(\tau)\cos\alpha_j(\tau)\sin\theta_j(\tau) \\ y_{d_j}(\tau+1) &= y_{d_j}(\tau) + m_j(\tau)\sin\alpha_j(\tau)\sin\theta_j(\tau) \\ z_{d_j}(\tau+1) &= z_{d_j}(\tau) + m_j(\tau)\cos\theta_j(\tau). \end{aligned} \tag{1}$$

Then, we analyze the factors that affect the maximum flying distance $m_j^{\max}(\tau)$ of the UAV d_j. Let $v_j(\tau)$ denote the flying velocity of the UAV d_j at time step τ, and when the UAVs' battery is sufficient, $m_j^{\max}(\tau) = v_j^{\max}\delta T$, where δT denotes the length of each interval. However, as mentioned above, each UAV eventually returns to its initial position P_j^{init}. Therefore, the UAV needs to guarantee adequate supplies of residual energy to turn back, which can be expressed as

$$\sum_0^\tau E_j(t) + E_j^{\text{return}}\left(||\omega_{d_j}(\tau) - P_j^{\text{init}}||\right) \leq E_j^{\text{cap}} \tag{2}$$

where $E_j^{\text{return}}(l)$ is the function related to the distance to return, and $E_j^{\text{return}}(||\omega_{d_j}(\tau) - P_j^{\text{init}}||)$ represents the amount of energy consumed to return to the initial position from the current position. It is noted that the pitch angle during the reentry period only makes a slight difference on energy consumption. Therefore, we set the rated power p^r and standard velocity v^r in return mode, and define $E_j^{\text{return}}(l) = p^r l / v^r$. Besides, E_j^{cap} is the capacity of the mounted battery, and $E_j^{\text{move}}(\tau)$ denotes the energy consumed by UAV's moving and hovering.

We consider $E_j(\tau)$ for a rotary-wing UAV whose energy consumption primarily consists of propulsion that generates acceleration and velocity [28]. Similar to [28], [29], and [30], without considering the acceleration, we define the power consumption of UAVs' flying and hovering with a velocity of $v_j(\tau)$ as [30]

$$\begin{aligned} p_j\big(v_j(\tau)\big) = {} & \frac{1}{2}d_0\rho_a s A v_j(\tau)^3 + F_j^f(\tau)v_j(\tau)\cos\theta(\tau) \\ & + p_b\left(1 + \frac{3v_j(\tau)^2}{V^2}\right) \\ & + p_i\left(\left(1 + \frac{v_j(\tau)^4}{4v_0^4}\right)^{\frac{1}{2}} - \frac{v_j(\tau)^2}{2v_0^2}\right)^{\frac{1}{2}} \end{aligned} \tag{3}$$

where p_b and p_i denote the blade profile power and induced power when $v_j(\tau) = 0$ (hovering), respectively. $F_j^f(\tau) = (1/2)\rho_a s_c v_j(\tau)^2 \cos\theta(\tau)$ is the UAV's thrust, where s_c represents the fuselag equipment cross-sectional area. According to (2), UAV's maximum movement distance $d_j^{\max}(\tau)$ can be

limited by

$$d_j^{\max}(\tau) \le \frac{1}{\frac{p(v_j(\tau))-p_j(0)}{v_j(\tau)} + \frac{p^r}{v^r}} \cdot \left(E_j^{\text{cap}} - \sum_{0}^{\tau-1} E_j(t) - p_j(0)\,\delta T - \frac{p^r ||\omega_{d_j}(\tau-1) - P_j^{\text{init}}||}{v^r} \right). \quad (4)$$

Therefore, $d_j^{\max}(\tau) = \min\{(4), v_j^{\max}\delta T\}$.

B. Communication Model

In our scenario, it exists both the ground-to-air (G2A) links and the ground-to-ground (G2G) links. For these two types of communication links, we consider the propagation path loss and the small-scale channel fading to constitute the channel gain.

In the G2A path-loss model, each IoTD has the probability to transmit LoS signal to the UAVs, and the LoS probability $P_{ij}^{\text{LoS}}(\tau)$ is calculated by

$$P_{ij}^{\text{LoS}}(\tau) = \frac{1}{1 + \lambda_1 e^{-\lambda_2(\theta_{ij}(\tau)-\lambda_1)}}, j \ge M_b \quad (5)$$

where λ_1 and λ_2 are the constants that depend on the environment (e.g., rural, suburban, urban, and dense urban), and $\theta_{ij}(\tau)$ is the elevation angle of u_i toward d_j. Let η_{LoS} and η_{NLoS} represent the excessive propagation losses related to the LoS and NLoS transmissions, respectively. Then, the G2A propagation path loss can be expressed as follows [31]:

$$\mathbb{L}_{ij}(\tau) = \frac{10^{-\frac{(\eta_{\text{LoS}}-\eta_{\text{NLoS}})P_{ij}^{\text{LoS}}(\tau)+\eta_{\text{NLoS}}}{10}}}{\left(||\omega_{d_j}(\tau) - \omega_{u_i}(\tau)||\right)^2 \left(\frac{4\pi f_c}{c}\right)^2} \ \forall i \in \mathcal{I}, j \ge M_b \quad (6)$$

where f_c and c denote the carrier frequency and the speed of light, respectively. In addition, for the G2G propagation path loss, we adopt the model in [32], which can be expressed as

$$\mathbb{L}_{ij}(\tau) = \left(\frac{c}{4\pi f_c}\right)^2 \left(\frac{||\omega_{d_j}(\tau) - \omega_{u_i}(\tau)||}{d_0}\right)^{-\eta_{\text{BS}}} \ \forall i \in \mathcal{I}, j < M_b \quad (7)$$

where η_{LoS} denotes the path-loss exponent, and d_0 is the reference distance.

Similar to [1], [33], and [34], we leverage the Nakagami-m model to describe the small-scale channel fading model. The probability density function of the small-scale channel fading $\mathbb{S}_{ij}(\tau)$ from u_i to d_j is expressed as

$$f_{\mathbb{S}_{ij}(\tau)}\left(\mathbb{S}_{ij}(\tau) = s\right) = \frac{m^{N_{ats}\cdot m} s^{N_{ats}\cdot m-1}}{(N_{ats}\cdot m - 1)!} e^{-ms} \ \forall i \in \mathcal{I} \ \ \forall j \in \mathcal{J} \quad (8)$$

where m represents the degree of fading, and N_{ats} is the number of antennas of the receiver (UAV or BS).

Then, we consider an orthogonal frequency-division multiple access (OFDMA) system, and all the MEC servers share a channel, which is divided into K subchannels [35]. The set of K subchannels is denoted by $\mathcal{K} = \{s_0, \ldots, s_k, \ldots, s_{K-1}\}$. Besides, one IoTD can only use one subchannel at the same time, while the multiple IoTDs can occupy the same subchannel to access different MEC servers [36], which can be expressed as follows:

$$\sum_i \sum_k a_{ij,k}(\tau) \le K \ \forall j \in \mathcal{J}$$
$$\sum_j \sum_k a_{ij,k}(\tau) \le 1 \ \forall i \in \mathcal{I} \quad (9)$$

where $a_{ij,k}(\tau)$ is the indicator variable. When u_i access the subchannel s_k on d_j, $a_{ij,k}(\tau) = 1$; otherwise, $a_{ij,k}(\tau) = 0$.

In addition, the IoTDs communicating with the MEC servers in the same subchannels exist the interference. Let $h_{ij}(\tau) = \mathbb{S}_{ij}(\tau)\mathbb{L}_{ij}(\tau)$ represent the channel gain of u_i to d_j at time step τ, and its single-to-interference ratio (SINR) in subchannel k can be expressed as

$$\gamma_{ij,k}(\tau) = \frac{p_i(\tau)h_{ij}(\tau)}{\left(\sum_{m\in\mathcal{J}} \sum_{l\in\mathcal{I}, l\ne i} a_{lm,k}(\tau)p_l(\tau)h_{lj}(\tau)\right) + \frac{B}{K}N_0} \quad (10)$$

where $p_i(\tau)$ denotes the transmit power of u_i, and B is the system total bandwidth. N_0 denotes the spectral density of the additive white Gaussian noise. According to (10), the IoTD's u_i uplink data transmission rate to the MEC server d_j on subchannel k is calculated as follows:

$$R_{ij,k}(\tau) = \frac{B}{K}\log\left(1 + \gamma_{ij,k}(\tau)\right). \quad (11)$$

It is evident that $R_i(\tau) = \sum_j \sum_k a_{ij,k}(\tau)R_{ij,k}(\tau)$. In addition, since the size of the resulted data from computation is usually slight, we neglect the downlink transmission rate and latency in this research work [6], [37], [38].

C. Task Offloading and Caching Queue

In this section, we introduce the task offloading model in the considered MEC network topology and the tasks cache queue updating on IoTDs. It is confirmed that the MEC servers with parallel computation capability can independently process offloaded tasks through virtualization technology [39]. Also, the central processing unit (CPU) mounted on the MEC servers has a high processing frequency compared with that on the IoTDs. Although the MEC servers have sufficient amount of memory, the CPU frequency $f_j^{Ser}(\tau)$ decreases with the sum of the IoTDs' offloaded task length $L_j^{\mathcal{B}(\tau)} = \sum_i \sum_k a_{ij,k}^{\tau} \sum_t \beta_i^t(\tau) l_i^t, t \in \{0, \ldots, \tau\}$. We denote by $L_j^{\max}$ the maximum amount of the processing data during a time step and consider a common pool of resources (CPRs) model [25], which can be expressed as

$$f_j^{Ser}(\tau) = \left(1 - \frac{L_j^{\mathcal{B}(\tau)}}{L_j^{\max}}\right) F_j \ \forall j \in \mathcal{J} \quad (12)$$

where F_j is the MEC server's overall CPU frequency. Also, we assume that the offloaded tasks on the MEC servers should be completed within maximum duration δT [40] to avoid the loss of the computation results due to the uncertain device association relationship at the next time step. The MEC server

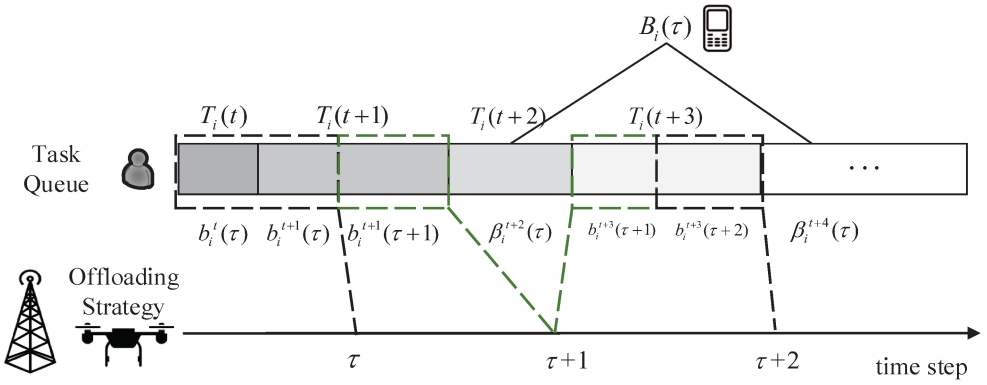


Fig. 2. Task offloading process of IoTDs includes local computing and offloading the task to the MEC server on BS/UAV.

d_j providing maximum CPU-Cycles during time step τ can be calculated by

$$C_j(\tau) = f_j^{Ser}(\tau)\left(\delta T - \max\left\{\frac{\sum_t^{\tau}\beta_i(t)l_i(t)}{R_{ij}(\tau)}\right\}\right). \quad (13)$$

Here, we consider that the MEC servers do not enter the parallel computation phase until completing all the tasks transmission. Then, the constraint of the offloaded tasks' required CPU-Cycles can be described as follows:

$$\sum_i\left(\sum_k\left(a_{ij,k}(\tau)\sum_t\phi_i(t)\beta_i(t)l_i(t)\right)\right) \le C_j(\tau)\ \forall j \in \mathcal{J}. \quad (14)$$

Meanwhile, the maximum offloading capacity to the MEC server of each IoTD is related to the transmission rate at the current time step τ, which can be expressed as

$$\sum_0^{\tau}\beta_i(t)l_i(t) \le \delta T R_i(\tau)\ \forall i \in \mathcal{I}. \quad (15)$$

The IoTDs serially process the remaining tasks in the queue on local without considering whether the current task could be completed within δT. However, since the tasks are indivisible, their uncompleted data fragment on local also cannot be offloaded. The local computation strategy is represented by $\mathcal{B}_i'(\tau) = \{b_i^0(\tau), \ldots, b_i^t(\tau), \ldots, b_i^{\tau}(\tau)\}, t \in [0, \tau]$, where $b_i(t)$ is the amount of processing tth task of IoTD u_i, which is a continuous variable. Thus, the amount of u_i's processing task is $L_i^{cpt}(\tau) = \sum_0^{\tau}(\beta_i(t)l_i(t) + b_i(t)), t \in [0, \tau]$, and the queue length $L_i(\tau)$ is updated by

$$L_i(\tau) = \max\left\{0, L_i(\tau-1) + l_i(\tau) - L_i^{cpt}(\tau)\right\}\ \forall i \in \mathcal{I}. \quad (16)$$

Fig. 2 displays an example of the computation and offloading program in the considered AIoT aerial MEC network. For convenience, the explanation of some important parameters is shown in Table II.

TABLE II
PARAMETER INTRODUCTION

Parameter	Detail Definition
$\mathbb{S}_{ij}(\tau)$	The small-scale fading from u_i to d_j
$f_{\mathbb{S}_{ij}(\tau}$	The probability density of $\mathbb{S}_{ij}(\tau)$
m	The degree of fading
N_{ats}	The number of antennas of the receiver(UAV or BS)
$\mathcal{K}$	The set of K subchannels
s_k	The k-th subchannel
$a_{ij,k}(\tau)$	If u_i has access to s_k on d_j
$h_{ij}(\tau)$	The channel gain of u_i to d_j at time step τ
$\gamma_{ij,k}(\tau)$	The SINR in s_k of u_i to d_j
$p_i(\tau)$	The transmit power of u_i
B	The system total bandwidth
N_0	The spectral density of the Additive White Gaussian Noise
$R_{ij,k}(\tau)$	The IoTD's u_i uplink data transmission rate to the MEC server d_j on subchannel k
$R_{ij}(\tau)$	The IoTD's u_i uplink data transmission rate to the MEC server d_j
$f_j^{Ser}(\tau)$	The CPU frequency of the MEC servers
$L_j^{\mathcal{B}(\tau)}$	The sum of the IoTs' offloaded task length
$C_j(\tau)$	The maximum CPU-Cycles provided by d_j during time step τ
$\beta_i^t(\tau)$	The offloading strategy
$b_i^t(\tau)$	The local computation strategy
$L_i^{cpt}(\tau)$	The amount of processing task of u_i

III. Problem Formulation

The MEC system aims at maximizing the overall QoE within T. The QoE is classified into two categories: 1) punctuality $\text{QoE}_\text{p}(\tau)$ and 2) task backlog $\text{QoE}b(\tau)$. $\text{QoE}_\text{p}(\tau)$ refers to the situation in which the task processing satisfies the latency requirement. For IoTD u_i, its completion time of the tth task is denoted by $t_i^t(\tau)$, which consists of queue blocking latency $t_i^{t,\text{block}}(\tau)$, computing latency, and transmission latency if offloading to the MEC servers. The response time $t_i^t(\tau)$ can be calculated as

$$t_i^t(\tau) = \begin{cases} \beta_i^t(\tau)\left(\frac{\sum_{s\in\mathcal{I}}\sum_k a_{sj^*,k}(\tau)\sum_{\mu\in[0,\tau]}\phi_s^\mu l_s^\mu\beta_s^\mu(\tau)}{f_{j^*}^{Ser}} + \frac{l_i(t)}{R_{ij^*}(t)}\right) \\ \quad + \frac{\phi_i^t b_i^t(\tau)}{f_i^{IoT}} + t_i^{t,\text{block}}(\tau), b_i^t(\tau) \text{ or } \beta_i^t(\tau) \neq 0 \\ +\text{inf, otherwise} \end{cases} \quad (17)$$

where j^* is the index of the MEC server connected with u_i, and the tth task of u_i is offloaded at time step τ^* or processed on local at time step τ'. However, either τ^* or τ' does not exist in the case that the task has not been processed till the current time step, and we set $t_i'(t)$ to positive infinity. Then, for each task $T_i(t)$, we define a latency satisfaction evaluation function $\psi(t_i^t)$, which can be expressed as [41]

$$\psi\left(t_i^t\right) = \begin{cases} 1, t_i^t \le t^{\text{opt}} \\ \frac{t^{\text{opt}}}{t^{\text{opt}}+t_i^t}, \text{ otherwise.} \end{cases} \tag{18}$$

At time step τ, the average latency satisfaction of all IoTDs for all generated tasks is denoted as $\text{QoE}_\text{p}(\tau)$. This metric is employed to assess the QoE provided by the system with respect to task latency satisfaction

$$\text{QoE}i^p(\tau) = \frac{1}{\tau}\sum_{t=0}^{\tau}\psi\left(t_i^t\right). \tag{19}$$

Analogously to $\text{QoE}_\text{p}(\tau)$, which signifies the average latency satisfaction, the average satisfaction regarding the cache queue length is denoted as $\text{QoE}b(\tau)$. This symbol encapsulates a specific physical interpretation, which is the ratio of the remaining cache queue capacity. It is articulated as follows:

$$\text{QoE}i^b(\tau) = \frac{L_i^{\max} - L_i(\tau)}{L_i^{\max}}. \tag{20}$$

Let $\mathcal{M}$, $\mathcal{A}$, and $\mathcal{B}$ represent, respectively, the strategies of UAV movement, device association, and offloading strategy across all time steps. Given these parameters, the optimization problem can be formulated as follows:

$$Q^*(\mathcal{M},\mathcal{A},\mathcal{B}) = \max_{\mathcal{M},\mathcal{A},\mathcal{B}} \sum_{\tau=0}^{T}\frac{1}{N}\sum_{i}\text{QoE}i^p(\tau)\cdot\text{QoE}i^b(\tau) \tag{21a}$$

$$\text{s.t. } \beta_i^t(\tau)\in\{0,1\}, \forall i\in\mathcal{I}, \tau=1,\ldots,T \tag{21b}$$

$$0\le b_i^t(\tau)\le 1, \forall i\in\mathcal{I}, \tau=1,\ldots,T \tag{21c}$$

$$0\le \beta_i^t(\tau)+b_i^t(\tau)\le 1 \ \forall i\in\mathcal{I}, \tau=1,\ldots,T \tag{21d}$$

$$x_l\le x_{d_j}(\tau)\le x_r \ \forall j\in\mathcal{J}, \tau=1,\ldots,T \tag{21e}$$

$$y_b\le y_{d_j}(\tau)\le y_f \ \forall j\in\mathcal{J}, \tau=1,\ldots,T \tag{21f}$$

$$z_d\le z_{d_j}(\tau)\le z_u \ \forall j\in\mathcal{J}, \tau=1,\ldots,T \tag{21g}$$

$$\text{(4), (9), (14), (15)} \tag{21h}$$

where x_l, x_r, y_b, and y_f are the horizontal boundary of the considered scenario, and the UAVs' altitude limit is between z_d and z_u. Equation (21b) represents that each task can only be completely offloaded or not offloaded.

IV. Problem Solution

In this section, we introduce the RSRT algorithm for IoTD association and the BTO algorithm for task offloading. Subsequently, we integrate both algorithms into the MADDPG algorithm, which is employed for UAV movement control in order to attain a comprehensive solution.

Algorithm 1 RSRT

Input: IoTD index i, Association strategy $\mathcal{A}(\tau)$
Output: $\mathcal{A}(\tau)$

Initialize: The alternative subchannel set $\mathcal{JK}_i(\tau)$ for u_i.
$j'k' = \arg\max_{jk\in\mathcal{JK}_i(\tau)}\{R_{ij,k}(\tau)\}$;
if the subchannel $j'k'$ is idle **then**
  $\mathcal{A}(\tau)[j'k'] = i$;
else
  if $R_{ij',k'} > R_{i^*j',k'}$ (u_i^* is the original IoTD in the subchannel $j'k'$) **then**
    $\mathcal{A}(\tau)[j'k'] = i$;
    Remove $j'k'$ from the set $\mathcal{JK}_i^*(\tau)$;
    Call **RSRT** $(i^*, \mathcal{A}(\tau))$;
  else
    Remove $j'k'$ from the set $\mathcal{JK}_i(\tau)$;
    Call **RSRT** $(i, \mathcal{A}(\tau))$;
  end if
end if

A. Device Association

We propose an RSRT algorithm to obtain the device association strategy $\mathcal{A}(\tau)$. Although the problem is an NP-hard problem and does not have the property of optimal substructure, this article utilizes the idea of greedy algorithms to access each IoTD to the allocated subchannel. In addition, UAVs act as base stations that can be moved around flexibly, and data can be temporarily cached in task queues as appropriate. Therefore, although IoTDs connected to subpar subchannels through greedy selection and replacement may currently experience suboptimal transmission rates, the proposed RSRT algorithm maximizes not only the total revenue at the current time but also makes it easier for IoTDs with lower individual benefits to be approached by UAVs through the subsequent UAV mobility control algorithm.

Specifically, for each IoTD u_i, access to MEC server j' is granted via subchannel k' denoted by $j'k'$, which ensures maximal transmission rate. However, multiple IoTDs may determine the same subchannel as optimal. In such cases, the IoTDs need to compare their transmission rates, and the IoTD with the highest transmission rate is granted access to the subchannel k'. The remaining IoTDs must search for the suboptimal subchannel using the same process. If the suboptimal subchannel is already occupied by another IoTD, then the two IoTDs involved in the contention must compare their transmission rates and make the decisions using recursive Algorithm 1.

B. IoTD Task Offloading

In this study, we aim to expedite the processing of task data by offloading the data to the base station. However, it is necessary to consider the latency of data transmission during offloading. Therefore, we investigate the means to achieve an optimal task offloading strategy that balances transmission and computing latency while minimizing the length of the task queue.

Despite the fact that the problem under consideration is classified as an NP-hard problem, it is possible to mitigate the computational complexity by limiting the solution space to a tractable range that is amenable to systematic exploration via the backtracking method. The application of backtracking to NP-hard problem involves several key factors that contribute to its effectiveness.

1) The ability to identify and apply appropriate pruning strategies that can reduce the size of the solution space and avoid exploring infeasible paths. This is often achieved by exploiting the problem structure and constraints to prune the search tree and eliminate unpromising branches early on. In the context of task offloading, several constraints may need to be considered. These constraints can impact the feasibility of backtracking on tasks offloading policy. Static constraint (15) pertains to the maximum data transmission volume that can be handled by each IoTD. Dynamic constraint (14) pertains to the maximum computing capacity of each base station. These constraints can impact the offloading decisions $\beta_i(t)$ made by the system and ensure that backtracking in the offloading process is both efficient and feasible.
2) The performance of backtracking is highly dependent on the choice of search order for variables and values, which can significantly impact the number of search nodes explored and the time complexity for obtaining an optimal solution. We focus on prioritizing offloading tasks with heavy data and long waiting time to the base station. This can improve the efficiency of data transmission and computation. Executing these tasks on local devices with limited resources results in increased computation latency and cache queue length. Thus, prioritizing offloading data-heavy and computationally intensive tasks can effectively improve the overall user experience. Further, the construction of a subset tree based on this principle for the solution space tree provides feasibility for the implementation of the backtracking method in NP-hard problems.

Then, we define $O_{i,u}(\tau)$ as the offloading amount of the IoTD i during time step τ after the task u is offloaded in the process of the algorithm. Let $T_j(\tau) = \{T_0(\tau), \ldots, T_u(\tau), \ldots, T_U(\tau)\}$, where u is the task in the IoTDs served by MEC server d_j, represent the set of offloaded tasks at time step τ. In addition, $L_{i,u}^{\text{remain}}(\tau)$ denotes the queue length of IoTD i after task u is offloaded. Moreover, tasks are deemed indivisible when offloaded to the base station. As a result, task offloading at each time step constitutes a 0-1 knapsack problem, in which the task data l_i^t corresponds to the weight of the item $\{w_{i,t}\}$, while $\{Q(O_{i,u}(\tau)) - Q(0)\}$ represents the value of the item $\{v_{i,t}\}$. To facilitate the efficient computation of upper bounds, items are only sorted by their value and subsequently assessed in that order with considering (15). For a group of IoTDs served by MEC server d_j, alternative tasks in their queues are treated as set $\mathbb{Q}_j(\tau) = \{v_{i,t}\}$. The offloaded tasks in set $\mathbb{Q}_j(\tau)$ are arranged in descending order based on the value $v_{i,t}$. To avoid taking too long in the sorting process and making the algorithm infeasible, we reduce the time complexity of the sorting algorithm from $o(n^2)$ to $o(2)$ by (33). Following this, a subset binary tree is constructed according to the order. For proof, refer to the Appendix, Lemma 1.

Algorithm 2 BTO

Input: The list *current_list*(τ) of tasks that are currently offloaded, the current value *current_value*(τ), the number of tasks remaining in the knapsack $n_{left}(\tau)$, the list max_*value_list*(τ) of task offloading with maximum value, the current maximum value max_*value*(τ), $\{w_{i,t}\}$, $\mathbb{Q}_j(\tau)$;

Output: $\mathcal{B}_j(\tau) \leftarrow$ max_*value_list*(τ)

```
if n_left(τ)==0 then
    if current_list(τ) ≥ max_value(τ) then
        max_value_list(τ) ← current_list(τ);
    end if
else
    temp ← current_list(τ);
    temp.append(n_left−1);
    Update Q_j(τ) according to (33);
    Call BTO (temp, max_value_list(τ), max_value(τ), n_left(τ)−1, {w_{i,t}}, Q_j(τ) );
    Call BTO (current_value(τ), max_value_list(τ), max_value(τ), n_left(τ)−1, {w_{i,t}}, Q_j(τ) )
end if
```

The proposed BTO algorithm is delineated in Algorithm 2. To minimize the stack space necessitated by parameter passing and recursive calls, we record the upper bound value and node information in the solution space tree while traversing each node of the binary tree. At the current extension node of the solution space tree, the upper bound value is computed solely when required to enter the right subtree for determining whether to prune it, while the upper bound value of the left child node remains identical to that of the parent node.

C. UAV Trajectory Planning

In this article, by applying the algorithm based on deep deterministic policy gradient (MADDPG), we utilize the feature extraction ability of deep learning and the decision ability of reinforcement learning to solve the multiagent cooperation problem. In general, the Markov process decision (MDP) is employed with the state space $\mathbb{S} = \{s^1, \ldots, s^\tau, \ldots, s^T\}$ and action space $\mathbb{A} = \{a^1, \ldots, a^\tau, \ldots, a^T\}$. As an agent, each UAV interacts with the environment and observes the state $s_j^\tau \in s^\tau$ at time step τ. In addition, the agent can exchange their private information with each other as well as obtaining a reward $r_j(\tau)$. Then, the environment transfers the state s^τ to the next state $s^{\tau+1}$ according to each action a_j^τ. Here, we independently execute and train the policy network and centrally train the Q network.

In addition, the actions of each UAV are not only affected by the environment but also related to other agents. Therefore, we design the state and action as follows.

1) *State* s_j^τ*:* We define the distance between d_j and u_i defined as $\|\omega_{d_j}(\tau) - \omega_{u_i}(\tau)\|$ as part of the observation. Moreover, the task queue of u_i is also taken into account.

2) *Action* a_j^τ*:* We consider the movement strategy of UAV d_j in the time slot τ as the action a_j^τ of the jth UAV.

The reward function for each d_j is defined as

$$r_j^\tau = \sum_{i \in I_j(\tau)} \frac{1}{\tau} \sum_{0}^{\tau} \psi\left(t_i'(t), t_i^{\max}(t)\right)\left(1 - \frac{L_i(\tau)}{L_i^{\max}}\right) - P_j(\tau) \quad (22)$$

where $P_j(\tau)$ is the penalty function, which is defined according to different constraints, respectively.

1) *UAVs Fly Out of the Determined Range:* When the UAVs perform communication task, some of them may fly out of the range. To avoid this problem, we should define a punishment mechanism

$$P_j(\tau) = \sum_j \upsilon_j \cdot \sqrt{\left(\chi_j^x(\tau)\right)^2 + \left(\chi_j^y(\tau)\right)^2}. \quad (23)$$

υ_j denotes the penalty coefficient. χ_j^x is the distance beyond the X-direction boundary, while χ_j^y is the distance beyond the Y-direction boundary at the time step τ. The expressions of calculation are as follows:

$$\chi_j^x(\tau) = \max\left(0, \left|x_{d_j}(\tau - 1) + m_j(\tau - 1)\cos\alpha_j(\tau - 1)\right| - |L|\right) \quad (24)$$

$$\chi_j^y(\tau) = \max\left(0, \left|y_{d_j}(\tau - 1) + m_j(\tau - 1)\sin\alpha_j(\tau - 1)\right| - |L|\right). \quad (25)$$

2) *UAVs Collision:* In order to avoid any UAVs colliding with each other, we have taken additional enforcement measures rather than just defining a penalty function P_j, as collisions are absolutely not allowed. Once a collision occurs during training, we add minus infinity from the reward and enter the next loop.

Then, the definitions of the entire state and action are as follows.

1) *State* s^τ*:* The state consists of the observations of $d_{M^b+1}, \ldots, d_{M_b+M_u-1}$, and it can be expressed as $s^\tau = \{o_j^\tau, \forall j \in [M^b + 1, M_b + M_u - 1]\}$.
2) *Action* a^τ*:* The action consists of the actions of $d_{M^b+1}, \ldots, d_{M_b+M_u-1}$, and it can be expressed as $a^\tau = \{a_j^\tau, \forall j \in [M^b + 1, M_b + M_u - 1]\}$.

Fig. 3 shows the multiagent cooperative based on the DRL algorithm framework, including policy network $\mu_j(s_j^\tau|\theta^{\mu_j})$, target policy network $\mu_j'(s_j^\tau|\theta^{\mu_j'})$, Q network $Q(s^\tau, a^\tau|\theta^{Q_j})$, and target Q network $Q'(s^\tau, a^\tau|\theta^{Q_j'})$. Particularly, each agent adopts the optimal action a_j^τ from the policy network $\mu_j(s_j^\tau|\theta^{\mu_j})$ and obtains the reward $r_j(\tau)$. Meanwhile, $Q(s^\tau, a^\tau|\theta^{Q_j})$ calculates Q to update the parameter θ^μ of policy network by policy gradient as follows:

$$\nabla_{\theta^{\mu_j}} J = \mathbb{E}\left[\nabla_{\theta^{\mu_j}} \mu_j\left(s_j^\tau|\theta^{\mu_j}\right) \nabla_{a_j^\tau} Q_j\left(s^\tau, a^\tau|\theta^{Q_j}\right)\right] \quad (26)$$

where $s^\tau = \{s_0^\tau, \ldots, s_j^\tau, \ldots, s_{M_u}^\tau\}$ and $s^\tau = \{a_0^\tau, \ldots, a_j^\tau, \ldots, a_{M_u}^\tau\}$. Then, each agent traverses to the next state $s_j^{\tau+1}$ and adopts the action $a_j^{\tau+1} = \mu_j'(s_j^{\tau+1})$. The target Q network outputs Q' to calculate target value y_τ

$$y_\tau = r_j^\tau + \gamma Q'\left(s^{\tau+1}, \mu_j'\left(s_j^{\tau+1}|\theta^{\mu_j'}\right)|\theta^{Q'}\right). \quad (27)$$

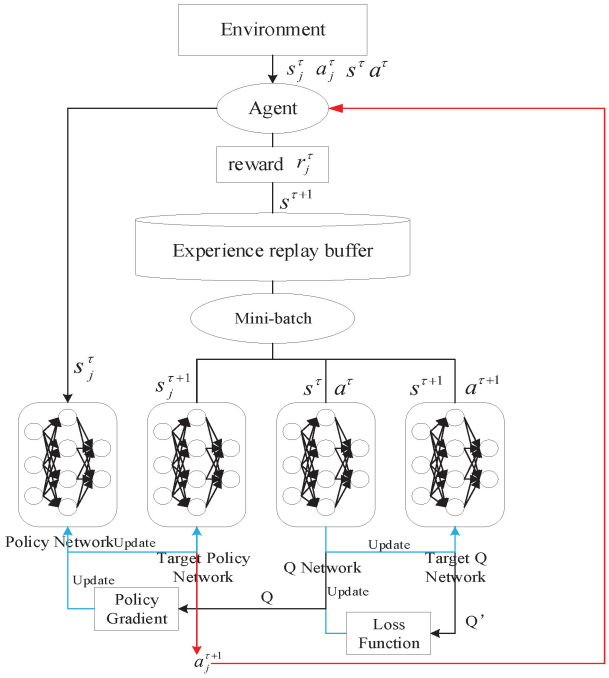


Fig. 3. MADDPG approach in controlling the movement of multiple UAVs.

In addition, we utilize an experience replay method to minimize the loss function of Q network with TD-error

$$L\left(\theta^{Q_j}\right) = \frac{1}{Z}\sum_z \left(y_\tau - Q_j\left(s^\tau, a^\tau|\theta^{Q_j}\right)\right)^2 \quad (28)$$

where Z is the number of sampling from the replay buffer.

V. Simulation

In this section, we present the extensive numerical evaluation to investigate the performance and intrinsic characteristics of the proposed framework in the aerial AIoT-enabled MEC network. Initially, various training parameters are examined to compare the training procedures, with the aim of identifying the optimal training parameters and their corresponding efficacy (Section V-A). Subsequently, the impact of communication parameters on the task offloading process is explored (Section V-B). In Section V-C, the comparative analysis between our methodology and alternative strategies is furnished. Also, we encapsulate the primary observations by offering insightful elucidations regarding the overall functionality and salient attributes of the proposed algorithm.

In the simulation, 50 IoTDs are randomly dispersed within a 500 m × 500 m area, with $(x_l, x_r) = (-250\text{ m}, 250\text{ m})$ and $(y_b, y_f) = (-250\text{ m}, 250\text{ m})$. The UAVs initiate their flight from the origin $(0, 0, 0)$ and ascend vertically to a height of $z_d = 50$ m prior to the commencement of the first time step in our time system. The maximum altitude constraint for UAVs is defined as $z_u = 150$ m. The IoTDs are served by two fixed-location BSs and four mobile UAVs together. For each IoTD, the length and the computing density of its task data generated per time step are uniformly distributed. In addition, the path-loss exponent between each IoTD and the GBS, denoted as η_{BS}, is set to 2, while the reference distance d_0 is established

TABLE III
SIMULATION PARAMETER INTRODUCTION

Parameter	Value	Parameter	Value
M^b	2	M^u	4
δT	2 s	E_j^{cap}	31680 kJ
F_j	$[4, 10] \times 10^9 Cycles/s$	f_j	$0.1 \times 10^9 Cycles/s$
B	20MHz	K	10
ϕ_i^t	[500,2000] Cycles/bit	l_i^t	[1000,5000] KB
p_i	0.2w	N_0	$10^{-17w/Hz}$
c	$3 \times 10^8 m/s$	f_c	4.9×10^9
λ_1, λ_2	4.88,0.43	η_{LoS}, η_{NLoS}	0.1, 21
N_{ats}	4(BS), 1(UAV)	L_j^{max}	50MB

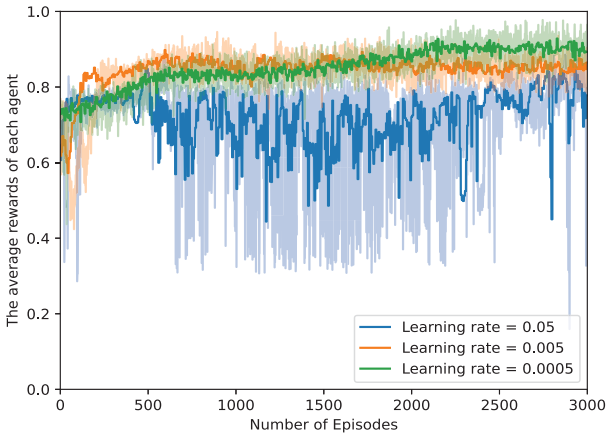


Fig. 4. Average reward of each agent versus the number of episodes under different learning rates.

at 1 m. The other details and parameter values are presented in Table III.

A. Training Procedures

By conducting a series of systematic experiments, we endeavor to identify the optimal values for two key parameters that impact the performance of the deep deterministic policy gradient (DDPG) algorithm: 1) learning rate and 2) discount factor. The experimental results, illustrated in Fig. 11, elucidate the convergence behavior of the reward function under varying learning rates. A learning rate of 0.05 proves to be excessively large, preventing convergence even after 3000 training iterations. Conversely, learning rates of 0.005 and 0.0005 enable convergence. Notably, the 0.005 learning rate fosters more rapid convergence, while a learning rate of 0.0005 results in superior final convergence performance for the reward function. The appropriate learning rate of 0.0005 is employed to evaluate the effects of distinct discount factors on the training performance. As illustrated in Fig. 5, a discount factor of 0.9 is identified as the most favorable option. The assignment of DRL parameters is presented in Table IV.

B. Simulation Result

In this section, we evaluate the impact of four different personal parameters in the considered scenario on the

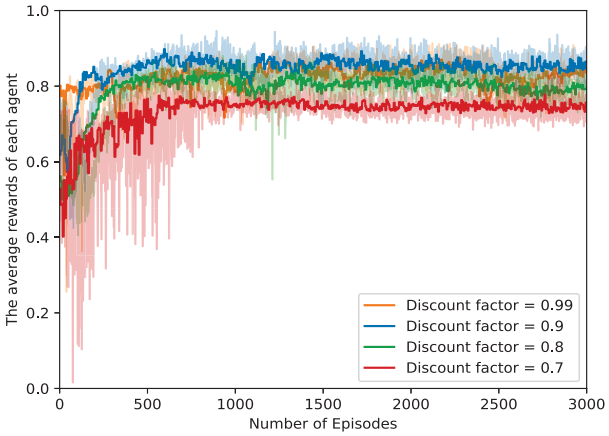


Fig. 5. Average reward of each agent versus the number of episodes under different discount factors.

TABLE IV
DDPG PARAMETER INTRODUCTION

Parameter	Value
Discount factor	0.9
Learning rate	0.0005
The number of episode	3000
Memory size	200000
Batch size	16384
The number of episode	3000

performance of IoTDs' task offloading strategy, including communication parameter (bandwidth B, transmit power p_i) and task parameter [length of data l_i^t, computing density $\phi_i^t(\tau)$]. To ensure the fairness of our simulation, we generate 1000 sets of task data, which included location information, and the length of task data at each time step. Then, we randomly select 50 or 100 groups of IoTDs for 100 times of verification and averaged the results to reduce the impact of randomness in the task data and the potential for bias. In our analysis, we examine the effect of each parameter individually, holding all other parameters constant at their baseline values. Specifically, we vary a single parameter while keeping all other parameters fixed.

Fig. 6 illustrates the impact of bandwidth on the objective function value and some communication indicators. The results indicate that increasing the total bandwidth provided to the base station improves the overall QoE performance. The reason is that a higher bandwidth allows for faster transmission rates, which reduces the transmission latency of offloading tasks from the local to the base station. Additionally, a larger amount of data can be transmitted during each time step, which can effectively reduce the cache queue length. It is worth noting that when the bandwidth changes from 10 to 15 MHz, the transmission latency increases because more tasks are offloaded. Accordingly, the waiting latency decreases, but the response time still decreases.

We conduct further experiments where we recreate the length of task data l as evenly distributed means. In Fig. 7(a), we plot the impact of l on the overall QoE performance. The results indicate that increasing the length of task data leads to

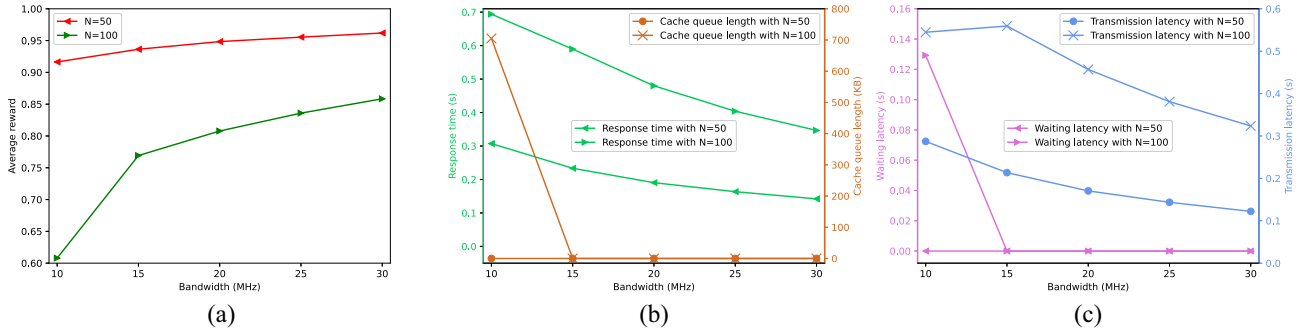


Fig. 6. Influence of bandwidth on reward function and some communication indicators. (a) Average reward versus bandwidth. (b) Response time and cache queue length. (c) Waiting latency and transmission latency versus bandwidth.

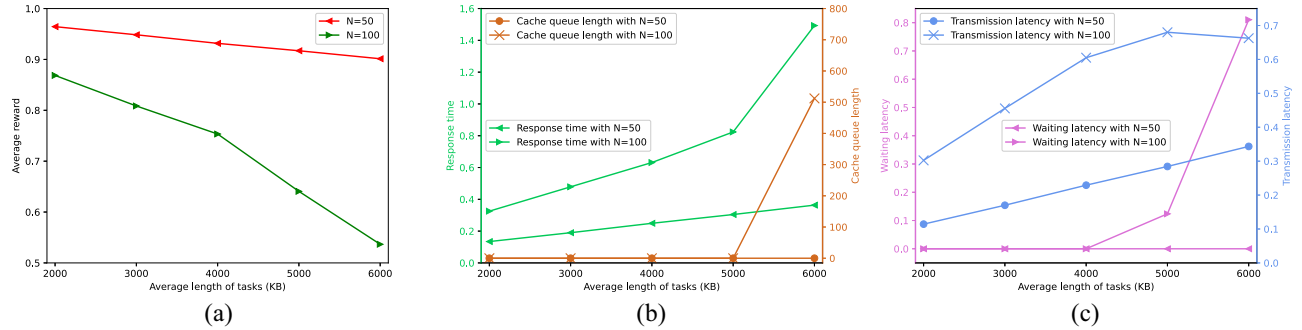


Fig. 7. Influence of average data length on reward function and communication indicators. (a) Average reward versus average data length. (b) Response time and cache queue length versus average data length. (c) Waiting latency and transmission latency versus average data length.

a significant decrease in QoE, particularly when the number of IoTDs is large. The reason is that when the transmission rate is constant, the increase in the length of data at each time step reduces the efficiency of task offloading. Moreover, when the number of IoTDs is large, the transmission rate decreases due to the increased intensity of interference signals. Therefore, the combined effect of increased data length and decreased transmission rate results in a significant increase in the transmission latency and waiting latency as shown in Fig. 7(b) and (c).

C. Comparison Simulation

In this section, we perform the evaluation of the proposed algorithm compared with four benchmark strategies or comparison algorithms as follows.

1) *Weighted K-Means With Constraint (WKMC):* Each IoTD is assigned a weight value based on its virtual queue length, which is expressed as follows:

$$L_i^v(t) = \sum_{\tau=0}^{t} e^{\frac{t_i^{t,\text{block}}(\tau)}{t_i^{t,\max}}} l_i^t(\tau). \tag{29}$$

Here, $L_i^v(t)$ describes the task waiting latency and the length of the virtual queue, which is normalized as the weight value. The weight value is between 0 and 1, where a higher weight value indicates a higher importance or relevance. During the clustering process, the weighted K-Means algorithm calculates the cluster centroids by taking into account the weight of each IoTD. Specifically, the algorithm computes the weighted mean of the data points belonging to each cluster to update the centroid of that cluster. It is worth noting that the distance between the updated centroid and the UAVs' position at the last time step is limited to $d_j^{\max}(\tau)$ during the iterative process. The iterative process continues until the algorithm converges to a stable clustering solution. The final result is a set of cluster centroids (UAVs' positions) and the IoTDs that belong to each cluster (device association).

2) *Fixed UAV Trajectory (FUT):* All UAVs fly in an orbit around a circle with a center of (0, 0) and a radius of $\sqrt{2}L/4$. Each UAV is capable of moving a maximum distance $m_j^{\max}/2$ at each time step τ, and the angle interval of each UAV is $2\pi/M^u$, where M^u represents the total number of UAVs.

3) *Nearest Association (NA) Algorithm:* For each u_i, it requests to connect with the nearest d_j. If the number of IoTDs served by d_j is less than K, d_j accepts the connection request. Otherwise, it requests to connect with the second nearest d_j' until it associates with a UAV or BS.

4) *WKMC+NA:* Utilize the weighted K-means clustering (WKMC) algorithm along with the NA algorithm. In this approach, the WKMC algorithm is used to optimize the position of UAVs, and the NA algorithm is used for device association.

Fig. 8 shows that as the number of IoTs increases, the value of the objective function decreases, with a rapid decline observed when the number of IoTs exceeds 70. The rapid increase in the number of connected IoTDs exacerbates

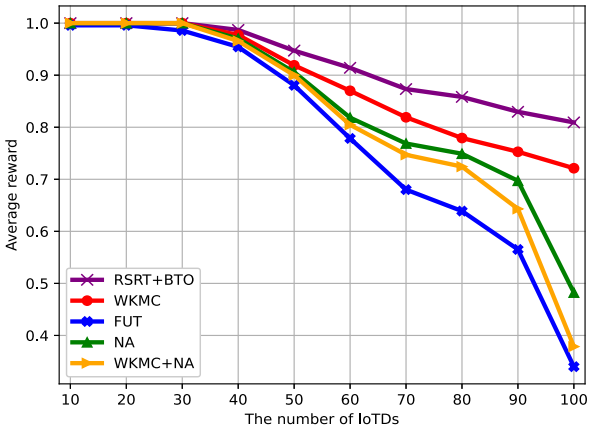
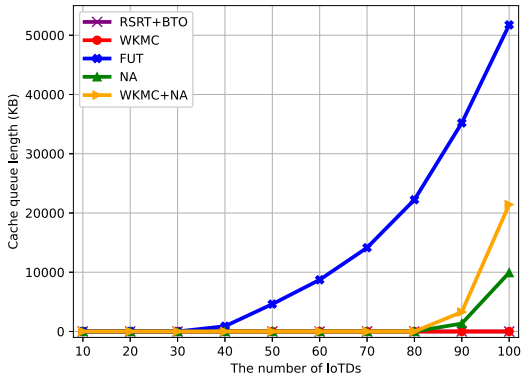
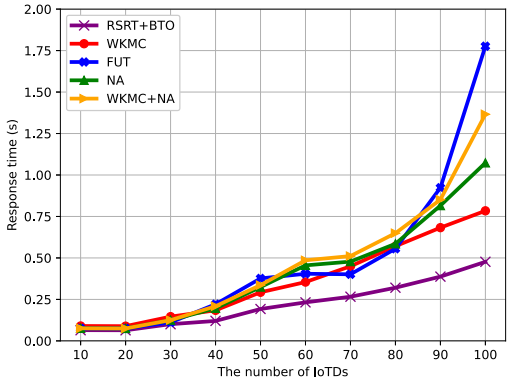


Fig. 8. Average reward versus the number of IoTDs for the proposed algorithms and four comparison algorithms.



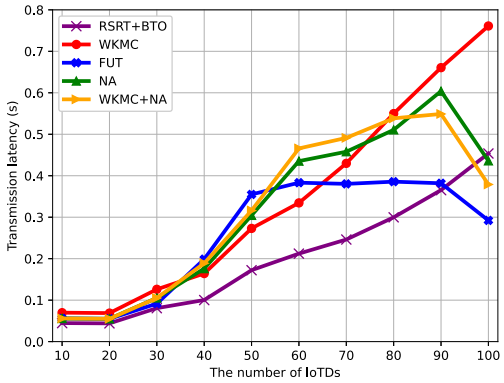
(a)



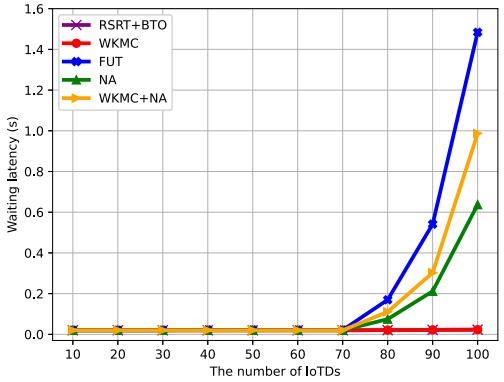
(b)

Fig. 9. (a) Response time and (b) cache queue length versus the number of IoTDs for the proposed algorithms and four comparison algorithms.

interference, which may result in some IoTDs not being served for a period of time or experiencing a significantly reduced transmission rate. However, the proposed RSRT+BTO algorithm and WKMC algorithm demonstrate improved adaptability in multi-IoTDs scenarios. Specifically, when $N = 100$, the proposed RSRT+BTO algorithm exhibits performance improvements of 2.347, 2.126, 1.723, and 1.110 times over FUT, WKMC+NA, NA, and WKMC, respectively. Fig. 9 provides an explanation for the decline in the objective function value. When the number of IoTDs in the FUT algorithm reaches 50, the number of remaining tasks begins to increase



(a)



(b)

Fig. 10. (a) Transmission latency and (b) waiting latency versus the number of IoTDs for the proposed algorithms and four comparison algorithms.

significantly. While the response time changes remain constant before reaching 80 IoTDs, this can be attributed to a large number of unprocessed tasks, which do not contribute to an increase in transmission latency. However, when the number of IoTDs reaches 100, the NA and WKMC+NA algorithms experience a significant increase in latency due to the large cache of tasks. As depicted in Fig. 10, these algorithms also exhibit a decrease in transmission latency and an increase in the waiting latency. In contrast, as the number of IoTDs increases, only the transmission latency increases for Algorithms RSRT+BTO and WKMC, while other parameters remain unchanged. This explains the lack of a significant drop in the objective function value. Nevertheless, our proposed RSRT and BTO algorithms demonstrate clear superiority over the WKMC algorithm in terms of response time and cache queue length.

In the ensuing comparative analysis, the task losses of five distinct algorithms are scrutinized when the variable M is set at the values 80, 90, and 100, respectively. As gleaned from the data encapsulated in Table V, it becomes evident that both our proposed algorithm and the WKMC algorithm are impervious to task losses. Conversely, the NA and WKMC+NA algorithms encounter minuscule task losses. Significantly, the FUT algorithm experiences a substantial number of task losses, substantiating the necessity and efficacy of UAV trajectory planning. Moreover, Fig. 11 serves to provide a comparative analysis on the fluctuation in cache queue size within an

TABLE V
TASK LOSS CONDITION

Algorithm \ Number of AIoTDs	80	90	100
RSRT+BTO	0	0	0
WKMC	0	0	0
FUT	4.786	14.94	21.35
NA	0	0.089	0.1
WKMC+NA	0	0.089	0.11

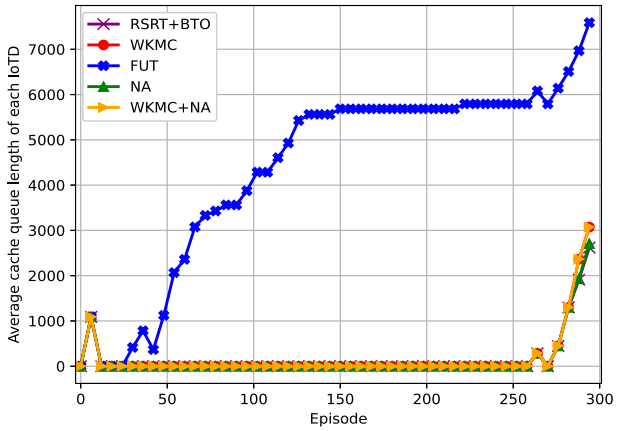


Fig. 11. Change of IoTDs' average cache queue length in 300 time steps under the same task length and computing density distribution of five algorithms when the number of IoTDs is 100.

episodic framework, which correspondingly aligns with the observed task losses. The observed consistency further bolsters our hypothesis concerning the crucial role of efficient UAV trajectory planning.

VI. CONCLUSION

This research presents a novel AI scheme for optimizing AIoT-enabled MEC networks through the integration of multi-UAVs. The scheme encompasses the RSRT algorithm for device association, the BTO algorithm for task offloading, and the MADDPG approach for UAV trajectory planning. The primary objective is to enhance the QoE for IoT devices by reducing response time and cache queue length. The experimental results demonstrate the effectiveness and superiority of the proposed approach over existing benchmark schemes. The scheme exhibits a high convergence rate and outperforms previous methods in optimizing response time and cache queue length for IoTDs. This research contributes to the advancement of AIoT-enabled aerial MEC networks by effectively addressing the challenges associated with device association, task offloading, and multi-UAV mobility through an integrated and innovative solution. Future research directions could involve further refining and enhancing the joint optimization scheme to improve the efficiency and effectiveness of aerial MEC networks.

APPENDIX

PROOF OF LEMMA 1

Our goal is to judge the positive impact of offloading a task u on the objective function. Therefore, we take $O_{i,u}(\tau)$ as variable and expand the problem (P1) to obtain the following formulation:

$$Q\big(O_{i,u}(\tau)\big) = \frac{L_{i,u}^{\text{remain}}(\tau) + O_{i,u}(\tau))}{L_i^{\max}} \cdot \frac{t^{\text{opt}}}{t^{\text{opt}} + \frac{O_{i,u}(\tau)}{R_i(\tau)}}. \quad (30)$$

Then, we estimate the effect of offloading a task u on the objective function, which can be expressed as $Q'(O_{i,u}(\tau)) = Q(O_{i,u}(\tau)) - Q(0)$, and $Q(0)$ is calculated as follows:

$$Q(0) = \frac{L_{i,u}^{\text{remain}}(\tau) - O_{i,u}(\tau)}{L_i^{\max}} \cdot \frac{t^{\text{opt}}}{t^{\text{opt}} + \delta T}. \quad (31)$$

Here, we assume that $Q(O_{i,u}(\tau))$ is a continuous variable and take the derivative with respect to $Q'(O_{i,u}(\tau))$

$$\begin{aligned}\frac{\partial Q'\big(O_{i,u}(\tau)\big)}{\partial O_{i,u}(\tau)} = {} & \frac{t^{\text{opt}}}{L_i^{\max}(\delta T + t^{\text{opt}})\big(R_i(\tau)t^{\text{opt}} + O_{i,u}(\tau)\big)^2} \cdot \\ & \times \big(R_i(\tau)^2 t^{\text{opt}}\big(\delta T + 2t^{\text{opt}}\big) + 2R_i(\tau)t^{\text{opt}}O_{i,u}(\tau) \\ & + O_{i,u}(\tau)^2 - L_{i,u}^{\text{remain}}(\tau)R_i(\tau)\big(\delta T + t^{\text{opt}}\big)\big). \quad (32)\end{aligned}$$

Then, we can obtain two extreme points $p_1(Q') = L_{i,u}^{\text{remain}} - R_i(\tau)\delta T - 2R_i(\tau)t^{\text{opt}} - \sqrt{(2R_i(\tau)t^{\text{opt}} + R_i(\tau)\delta T - L_{i,u}^{\text{remain}})^2 - 4L_{i,u}^{\text{remain}}R_i(\tau)\delta T}$ and $p_2(Q') = L_{i,u}^{\text{remain}} - R_i(\tau)\delta T - 2R_i(\tau)t^{\text{opt}} + \sqrt{(2R_i(\tau)t^{\text{opt}} + R_i(\tau)\delta T - L_{i,u}^{\text{remain}})^2 - 4L_{i,u}^{\text{remain}}R_i(\tau)\delta T}$. In addition, when $O_{i,u}(\tau)$ approach to $+\inf$, $Q'(O_{i,u}(\tau)) = +\inf$. On the contrary, when $O_{i,u}(\tau)$ approach to $-\inf$, $Q'(O_{i,u}(\tau)) = -\inf$. Therefore, the function $Q'(O_{i,u}(\tau))$ increases and then decreases and then increases, which extreme points are $p_1(Q')$ and $p_2(Q')$, respectively. Thus, the subset tree sorting selection for each episode in the traceback method is as follows:

$$\begin{cases} \max\big\{O_{i,u}(\tau)\big\}, p_1\big(Q'\big), p_2\big(Q'\big) \le 0 \\ \arg\max\big\{Q'(0), Q'\big(\max\big\{O_{i,u}(\tau)\big\}\big)\big\}, \text{otherwise.} \end{cases} \quad (33)$$

REFERENCES

[1] A. Boumaalif and O. Zytoune, "Power distribution of device-to-device communications under Nakagami fading channel," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2158–2167, Jun. 2022.
[2] J. Wang, K. Liu, and J. Pan, "Online UAV-mounted edge server dispatching for mobile-to-mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1375–1386, Feb. 2020.
[3] S. K. Kasi et al., "Heuristic edge server placement in Industrial Internet of Things and cellular networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10308–10317, Jul. 2021.
[4] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.
[5] M. Mukherjee, V. Kumar, A. Lat, M. Guo, R. Matam, and Y. Lv, "Distributed deep learning-based task offloading for UAV-enabled mobile edge computing," in *Proc. 39th IEEE Conf. Comput. Commun.*, 2020, pp. 1208–1212.
[6] M. Li, N. Cheng, J. Gao, Y. Wang, L. Zhao, and X. Shen, "Energy-efficient UAV-assisted mobile edge computing: Resource allocation and trajectory optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3424–3438, Mar. 2020.
[7] R. Han, Y. Wen, L. Bai, J. Liu, and J. Choi, "Rate splitting on mobile edge computing for UAV-aided IoT systems," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1193–1203, Dec. 2020.
[8] G. Fragkos, E.-E. Tsiropoulou, and S. Papavassiliou, "Artificial intelligence enabled distributed edge computing for Internet of Things applications," in *Proc. 16th Int. Conf. Distrib. Comput. Sens. Syst.*, 2020, pp. 450–457.

[9] D. Wei, J. Ma, L. Luo, Y. Wang, L. He, and X. Li, "Computation offloading over multi-UAV MEC network: A distributed deep reinforcement learning approach," *Comput. Netw.*, vol. 199, Nov. 2021, Art. no. 108439.

[10] S. Shi, M. Wang, S. Gu, and Z. Zhong, "Energy-efficient UAV-enabled computation offloading for Industrial Internet of Things: A deep reinforcement learning approach," *Wireless Netw.*, vol. 2021, pp. 1–10, Oct. 2021.

[11] Y. Nie, J. Zhao, F. Gao, and F. R. Yu, "Semi-distributed resource management in UAV-aided MEC systems: A multi-agent federated reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13162–13173, Dec. 2021.

[12] Y. Ye, W. Wei, D. Geng, and X. He, "Dynamic coordination in UAV swarm assisted MEC via decentralized deep reinforcement learning," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, Nanjing, China, Oct. 2020, pp. 1064–1069.

[13] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.

[14] Y. Liu, S. Xie, and Y. Zhang, "Cooperative offloading and resource management for UAV-enabled mobile edge computing in power IoT system," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12229–12239, Oct. 2020.

[15] Y. Miao, K. Hwang, D. Wu, Y. Hao, and M. Chen, "Drone swarm path planning for mobile edge computing in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 5, pp. 6836–6848, May 2023.

[16] L. P. Qian, H. Zhang, Q. Wang, Y. Wu, and B. Lin, "Joint multi-domain resource allocation and trajectory optimization in UAV-assisted maritime IoT networks," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 539–552, Jan. 2023.

[17] Q. Luo, T. H. Luan, W. Shi, and P. Fan, "Deep reinforcement learning based computation offloading and trajectory planning for multi-UAV cooperative target search," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 504–520, Feb. 2023.

[18] J. Bai, Z. Zeng, T. Wang, S. Zhang, N. N. Xiong, and A. Liu, "TANTO: An effective trust-based unmanned aerial vehicle computing system for the Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5644–5661, Apr. 2023.

[19] J. Bai, G. Huang, S. Zhang, Z. Zeng, and A. Liu, "GA-DCTSP: An intelligent active data processing scheme for UAV-enabled edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 4891–4906, Mar. 2023.

[20] P. Chen et al., "Secure task offloading for MEC-aided-UAV system," *IEEE Trans. Intell. Veh.*, vol. 8, no. 5, pp. 3444–3457, May 2023.

[21] Y.-J. Chen, K.-M. Liao, M.-L. Ku, F. P. Tso, and G.-Y. Chen, "Multi-agent reinforcement learning based 3D trajectory design in aerial-terrestrial wireless caching networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8201–8215, Aug. 2021.

[22] H. Yan, W. Bao, X. Zhu, J. Wang, and L. Liu, "Data offloading enabled by heterogeneous UAVs for IoT applications under uncertain environments," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 3928–3943, Mar. 2023.

[23] D. S. Lakew, A.-T. Tran, N.-N. Dao, and S. Cho, "Intelligent offloading and resource allocation in heterogeneous aerial access IoT networks," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5704–5718, Apr. 2023.

[24] Z. Yang, S. Bi, and Y.-J. A. Zhang, "Dynamic offloading and trajectory control for UAV-enabled mobile edge computing system with energy harvesting devices," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10515–10528, Dec. 2022.

[25] P. A. Apostolopoulos, G. Fragkos, E. E. Tsiropoulou, and S. Papavassiliou, "Data offloading in UAV-assisted multi-access edge computing systems under resource uncertainty," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 175–190, 1 Jan. 2023.

[26] Y. Jin and H. J. Lee, "On-demand computation offloading architecture in fog networks," *Electronics*, vol. 8, no. 10, p. 1076, 2019.

[27] J. Wang, D. Feng, S. Zhang, J. Tang, and T. Q. S. Quek, "Computation offloading for mobile edge computing enabled vehicular networks," *IEEE Access*, vol. 7, pp. 62624–62632, 2019.

[28] N. Gupta, D. Mishra, and S. Agarwal, "Energy-aware trajectory design for outage minimization in UAV-assisted communication systems," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1751–1763, Sep. 2022.

[29] B. Khamidehi and E. S. Sousa, "Reinforcement-learning-aided safe planning for aerial robots to collect data in dynamic environments," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13901–13912, Aug. 2022.

[30] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.

[31] S. Huang, L. Li, Q. Pan, W. Zheng, and Z. Lu, "Fine-grained task offloading for UAV via MEC-enabled networks," in *Proc. IEEE 30th Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC Workshops)*, Istanbul, Turkey, Sep. 2019, pp. 1–6.

[32] In T. J. Rouphael, *RF and Digital Signal Processing for Software-Defined Radio*. Burlington, NJ, USA: Newnes, 2009, pp. 377–383.

[33] I. Atzeni, J. Arnau, and M. Kountouris, "Downlink cellular network analysis with LOS/NLOS propagation and elevated base stations," *IEEE Trans. Wireless Commun.*, vol. 17, no. 1, pp. 142–156, Jan. 2018.

[34] B. Tahir, S. Schwarz, and M. Rupp, "Analysis of uplink IRS-assisted NOMA under Nakagami-m fading via moments matching," *IEEE Wireless Commun. Lett.*, vol. 10, no. 3, pp. 624–628, Mar. 2021.

[35] B. Di, L. Song, and Y. Li, "Sub-channel assignment, power allocation, and user scheduling for non-orthogonal multiple access networks," *IEEE Trans. Wireless Commun.*, vol. 15, no. 11, pp. 7686–7698, Nov. 2016.

[36] L. Liu, B. Sun, X. Tan, and D. H. K. Tsang, "Energy-efficient resource allocation and subchannel assignment for NOMA-enabled multiaccess edge computing," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1558–1569, Mar. 2022.

[37] E. Xuefei, Z. Ma, and J. F. Huang, "Cluster design and optimization of SWIPT-based MEC networks with UAV assistance," *Wireless Commun. Mobile Comput.*, vol. 2021, pp. 1–4, Nov. 2021.

[38] Y. K. Tun, Y. M. Park, N. H. Tran, W. Saad, S. R. Pandey, and C. S. Hong, "Energy-efficient resource management in UAV-assisted mobile edge computing," *IEEE Commun. Lett.*, vol. 25, no. 1, pp. 249–253, Jan. 2021.

[39] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "Mobile edge computing: Survey and research outlook," *IEEE Commun. Surveys Tuts.*, submitted to publication.

[40] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.

[41] S. Lee and H. Kim, "ACO-based optimal node selection method for QoE improvement in MEC environment," in *Proc. Int. Conf. Electron., Inf., Commun. (ICEIC)*, 2019, pp. 1–4.